

IFGOIANO – CAMPUS URUTAÍ SI		 INSTITUTO FEDERAL GOIANO Campus Urutaí
ED I	3º Matutino	Júnio César de Lima
Lista de Exercícios 2 Lista estática		

Exercício 1) Quando (em quais situações) usar alocação de memória estática e quando usar alocação de memória dinâmica?

Exercício 2) Na operação de remoção no início, por que geralmente deslocamos os itens para a esquerda em vez de apenas mover o índice 'primeiro'?

Exercício 3) Implemente em Java a classe Vetor.

Vetor
-elementos: String [] -tamanho: int
+Vetor() +Vetor(capacidade:int) +adiciona(elemento:String): boolean +adiciona(posicao:int, elemento:String): boolean +busca(posicao:int): String +busca(elemento:String): int +tamanho(): int +remove(posicao:int): void +toString(): String

Onde,

Vetor() é o construtor que inicializa o tamanho do vetor com TAM_MAX = 100.

Vetor(capacidade: int) é o construtor que inicializa o vetor com tamanho capacidade.

adiciona(elemento: String): boolean

insere elemento no final da lista, isto é, na primeira posição livre do vetor elementos[].

Retorna true (sucesso) ou false

adiciona(posicao:int, elemento: String): boolean

insere elemento na posição posicao do vetor elementos[]. Retorna true (sucesso) ou false

busca(posicao: int): String

retorna o String que está na posição posicao do vetor elementos[]

busca(elemento: String): int

busca pelo elemento no vetor elementos[] e retorna a sua posição, ou o valor -1 se não encontrar.

tamanho(): int

retorna a quantidade de elementos inseridos em elementos[].

remove(posicao: int)

remove o String da posição posicao.

ToString(): String

retorna um String com os elementos da lista.

Exercício 4) Qual é a principal desvantagem do uso de vetores para implementar listas lineares em comparação com listas encadeadas?

Exercício 5) Implemente um método que receba um número inteiro como parâmetro e retorne quantas vezes ele aparece na lista.

Exercício 6) Crie uma classe de teste para adicionar elementos no vetor (exercício anterior) e imprimir o vetor.

Exercício 7) Considere uma lista contendo números inteiros. Escreva um método **public Lista extrairPares()** que crie e retorne uma nova Lista contendo apenas os números pares da lista original. A lista original não deve ser modificada. Se não houver pares, retorne uma lista vazia.

Exercício 8) Escreva um método **public int contarOcorrencias(Object elemento)** que receba um elemento como parâmetro e retorne quantas vezes ele aparece na lista.

Exercício 9) Crie um método **public int substituirTodos(Object velho, Object novo)** que procure por todas as ocorrências do item velho e as substitua pelo item novo. O método deve retornar um número inteiro informando quantas substituições foram realizadas.

Exercício 10) Implemente o método **public Lista subLista(int inicio, int fim)**. Ele deve retornar uma nova Lista contendo os elementos do índice inicio até o índice fim - 1. O método deve validar se os índices passados são válidos.

Exercício 11) Considere que a lista armazena números inteiros. Crie um método **public void removerNegativos()** que remova todos os números menores que zero da própria lista, sem criar uma lista auxiliar.

Exercício 12) Escreva um método **public void concatenar(Lista l2)** que receba uma segunda lista l2 e adicione todos os elementos dela ao final da lista atual. Se o array da lista atual não tiver espaço suficiente para todos os elementos de l2, o método deve inserir até onde couber e parar.

Exercício 13) Escreva um método **public void inverter()** que inverta a ordem dos elementos da lista atual. O primeiro elemento passa a ser o último, o segundo passa a ser o penúltimo, e assim por diante. A inversão não pode instanciar uma nova Lista ou um novo vetor auxiliar.

Exercício 14) Escreva um método **public Lista intercalar(Lista l2)** que retorne uma terceira lista contendo os elementos da lista atual e da lista l2 intercalados (um da primeira, um da segunda, um da primeira...). Se uma lista for maior que a outra, os elementos restantes da lista maior devem ser adicionados ao final da nova lista.

Exercício 15) Implemente o método **public void removerDuplicatas()**. Este método deve varrer a lista e, sempre que encontrar um elemento que já apareceu anteriormente, deve removê-lo, mantendo apenas a primeira ocorrência de cada item. Ao final, a lista deve ter seu tamanho ajustado corretamente e não ter "buracos" no vetor.

Exercício 16) Suponha que a lista armazene caracteres (ou strings de uma letra). Crie um método **public boolean isPalindromo()** que retorne true se a sequência de elementos da lista for a mesma quando lida da esquerda para a direita e da direita para a esquerda, e false caso contrário.